

Author

Name : Mudireddy Krishna Vaishnavi

Roll Number : 24F2009315

Email : 24f2009315@ds.study.iitm.ac.in

About Me:

Student at IIT Madras BS in Data Science and Applications, interested in building real-world web apps that improve daily life through simple, efficient design.

Description

HealthEase is a hospital management web application that helps administrators, doctors, and patients manage appointments, medical records, and profiles through a unified system. It ensures smooth scheduling, secure access, and easy retrieval of treatment history.

Technologies Used

- **Backend:** Flask (Python)
- **Frontend:** HTML, CSS, Bootstrap, Jinja2
- **Database:** SQLite (auto-created)
- **Flask Extensions:** Flask-Login, Flask-SQLAlchemy, Flask-WTF, WTForms

Purpose:

Flask provides a lightweight yet scalable backend. SQLite offers simplicity and portability. Flask-Login and SQLAlchemy enable secure authentication and ORM-based DB operations. Bootstrap ensures responsive UI.

DB Schema Design

Tables:

1. **User:** `user_id`, `name`, `username`, `password`, `role`
2. **Doctor:** `doctor_id`, `specialization`, `availability`, `user_id` (FK)
3. **Patient:** `patient_id`, `user_id` (FK), `age`, `gender`, `contact`
4. **Appointment:** `appointment_id`, `doctor_id`, `patient_id`, `date`, `time`, `status`
5. **Treatment:** `appointment_id` (FK), `diagnosis`, `prescription`, `notes`

Reasoning:

The design ensures one-to-many relationships (Doctor–Appointment–Treatment), eliminates redundancy, and prevents scheduling conflicts.

API Design

APIs provide JSON data for doctors, patients, and appointments.

Example:

- `GET /api/doctors` – List all doctors
- `POST /api/appointments` – Book appointment
- `PUT /api/appointments/<id>` – Update status

Implemented using Flask routes returning JSON responses for future integration or testing.

Architecture and Features

Structure:

- `app.py` → Main application logic and routes
- `models.py` → Database models and relationships
- `templates/` → HTML (Jinja2)
- `application/` → Individual Routes for roles and appointments

Key Features:

- Role-based login (Admin, Doctor, Patient)
- CRUD operations for users and appointments
- Appointment conflict prevention
- Doctor treatment history tracking
- Responsive UI and validation

Additional Features

- Optional Chart.js for admin analytics, backend & frontend form validation.

Video

 2025-10-30_17-39-02.mkv

<https://drive.google.com/file/d/1tgE6L2REwMovCwr6BmdMNwjgP1ZH3JyG/view?usp=sharing>